МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра интеллектуальных и киберфизических систем

Лабораторная работа №5

«Создание проекта и подключение к системе контроля версий. Итеративный подход в разработке. Формирование комплекта программной и эксплуатационной документации»

Проект: «HammerLive» – информационная система для проведения онлайн-аукционов в реальном времени

Выполнили:
Студент 4 курса, 5 группы
Сиренко Иван Владиславович
Студент 4 курса, 5 группы
Гардееня Артем Сергеевич
Студентка 4 курса 5 группы
Голуб Александра Владимировна

Проверил:
преподаватель
Ломако Алексей Андреевич

Минск, 2026

ВВЕДЕНИЕ

В работе материалы проекта HammerLive, созданные в лабораторных работах №1–4, перенесены в систему контроля версий Git и размещены на GitHub. По итогам второй итерации выделены четыре замечания к системе, на их основе актуализированы UML-диаграммы, архитектура и спецификация API, изменения оформлены в отдельной ветке и Pull Request. Все документы сведены в единый комплект программной и эксплуатационной документации версии 1.1.

Ссылка на репозиторий: https://github.com/Gardar42/hammerlive

Pull Request итерации 2: https://github.com/Gardar42/hammerlive/pull/1

РАЗДЕЛ 1. ОРГАНИЗАЦИЯ ПРОЕКТА В СИСТЕМЕ КОНТРОЛЯ ВЕРСИЙ

1.1. Структура репозитория

Репозиторий разделён на папки по типу содержимого, чтобы документация, проектные материалы, код и конфигурация развивались независимо и не смешивались (рисунок 1).

```
hammerlive/
├── README.md           описание, сборка, информация для разработчиков
├── CHANGELOG.md        журнал изменений по версиям
├── .gitignore          исключения для Python, Node.js, Docker, IDE
├── .github/            шаблон описания Pull Request
├── api/openapi.yaml    спецификация API (OpenAPI 3.0)
├── config/             docker-compose.yml, .env.example, k8s/
├── design/
│   ├── uml/            UML-диаграммы (PlantUML, SVG, PNG)
│   ├── er/             ER-диаграммы (SVG, PNG)
│   ├── architecture/   диаграмма паттерна CQRS
│   └── prototypes/     вайрфреймы интерфейса (HTML + PNG)
├── docs/
│   ├── HammerLive_Комплект_документации_v1.1.pdf
│   ├── feedback/       замечания итерации 2
│   └── labs/           отчёты ЛР №1–5 (PDF, исходники в src/)
└── src/
    ├── services/       13 микросервисов (app/, tests/, Dockerfile)
    └── web-client/     веб-клиент (TypeScript, React)
```

Рисунок 1 – Структура репозитория

Исходники диаграмм хранятся в текстовом виде: PlantUML-описания и Python-скрипты, формирующие SVG, поэтому изменения в них видны в истории Git построчно, а PNG-файлы генерируются из исходников. Состав микросервисов соответствует диаграмме компонентов ЛР №3 и пяти архитектурным квантам: «Аукционист» (auction-session, auctioneer-capture), «Участник» (bid-capture), «Потоки участников» (bid-streamer, video-streamer, video-capture), «Отслеживание ставок» (bid-tracker) и «Платёж» (payment); кроме них созданы api-gateway и вспомогательные identity, reputation, notification, а в итерации 2 – telegram-adapter. Для каждого сервиса создана заготовка: приложение FastAPI с эндпоинтом /health, тест, файл зависимостей и Dockerfile. В config/docker-compose.yml описано локальное окружение из

PostgreSQL (отдельная база для каждого сервиса, хранящего данные), Redis, брокера сообщений RabbitMQ и всех сервисов.

1.2. Файлы README.md и .gitignore

Файл README.md в корне репозитория содержит краткое описание проекта на основе резюме ТЭО, ссылку на техническое задание (актуальная версия 1.1 – часть 1 комплекта документации, исходная версия 1.0 – в docs/labs), описание структуры репозитория, предполагаемые шаги сборки и запуска (установка Docker, Python и Node.js, настройка переменных окружения, запуск docker compose, миграции БД, запуск веб-клиента) и информацию для разработчиков: используемый стек (Python 3.12 и FastAPI, PostgreSQL 16 с отдельной базой у каждого сервиса, Redis 7, RabbitMQ, TypeScript и React), правила именования веток и порядок работы через Pull Request.

Файл .gitignore настроен под выбранный стек: исключаются кэш и виртуальные окружения Python (__pycache__, .venv, .pytest_cache), зависимости и сборка Node.js (node_modules, dist), файлы с секретами (.env, config/.env, *.secret.yaml), служебные файлы IDE и ОС, а также временные файлы Word (~$*), которые появляются при редактировании документов.

1.3. История коммитов

Ветка main содержит исходное состояние проекта, ветка feature/iteration-2-feedback – изменения итерации 2 (таблица 1). Прямые изменения в main не вносятся: все доработки проходят через Pull Request.

Таблица 1 – Коммиты репозитория

Ветка	Коммит
main	Инициализация репозитория: README, .gitignore, шаблон PR
main	Добавлены документы ЛР 1–4: ТЭО, ТЗ v1.0, UML-диаграммы, прототипы, модель данных, API v1.0
main	Заготовки микросервисов по архитектурным квантам, веб-клиента и конфигурация docker-compose
feature/iteration-2-feedback	FB-1, FB-2, FB-4: актуализированы диаграммы вариантов использования, компонентов и последовательностей (v1.1)
feature/iteration-2-feedback	FB-1 – FB-4: спецификация API v1.1, модель данных и заготовка сервиса telegram-adapter
feature/iteration-2-feedback	Комплект программной и эксплуатационной документации v1.1

РАЗДЕЛ 2. ИТЕРАТИВНАЯ ДОРАБОТКА ПРОЕКТНЫХ ДОКУМЕНТОВ

2.1. Замечания и новые требования

По результатам тестирования прототипа интерфейса и обратной связи заказчика выделены четыре замечания (таблица 2). Они сохранены в репозитории в файле docs/feedback/iteration-2.md, и на их идентификаторы ссылаются коммиты, Pull Request и изменённые пункты документов.

Таблица 2 – Замечания итерации 2

ID	Замечание / требование	Источник
FB-1	Участники, которые не могут присутствовать на торгах, должны иметь возможность заранее оставить заочную ставку	Заказчик
FB-2	Крупные ставки должны подтверждаться одноразовым кодом – защита от ошибочных ставок и захвата аккаунта	Тестирование прототипа
FB-3	Задержка обработки ставки – не более 150 мс (p95) вместо 200 мс, до 10 000 участников в одном аукционе	Заказчик
FB-4	Уведомления в Telegram – обязательная функция MVP, а не желательная	Опрос бета-тестеров

2.2. Изменения UML-диаграмм

На диаграмму вариантов использования (рисунок 2) добавлены три новых варианта: «Оставить заочную ставку» (FB-1), включающий холдирование депозита, «Подтвердить ставку одноразовым кодом» (FB-2), расширяющий «Сделать ставку» при условии [сумма > порога], и «Подключить Telegram-уведомления» (FB-4) с новым действующим лицом Telegram.

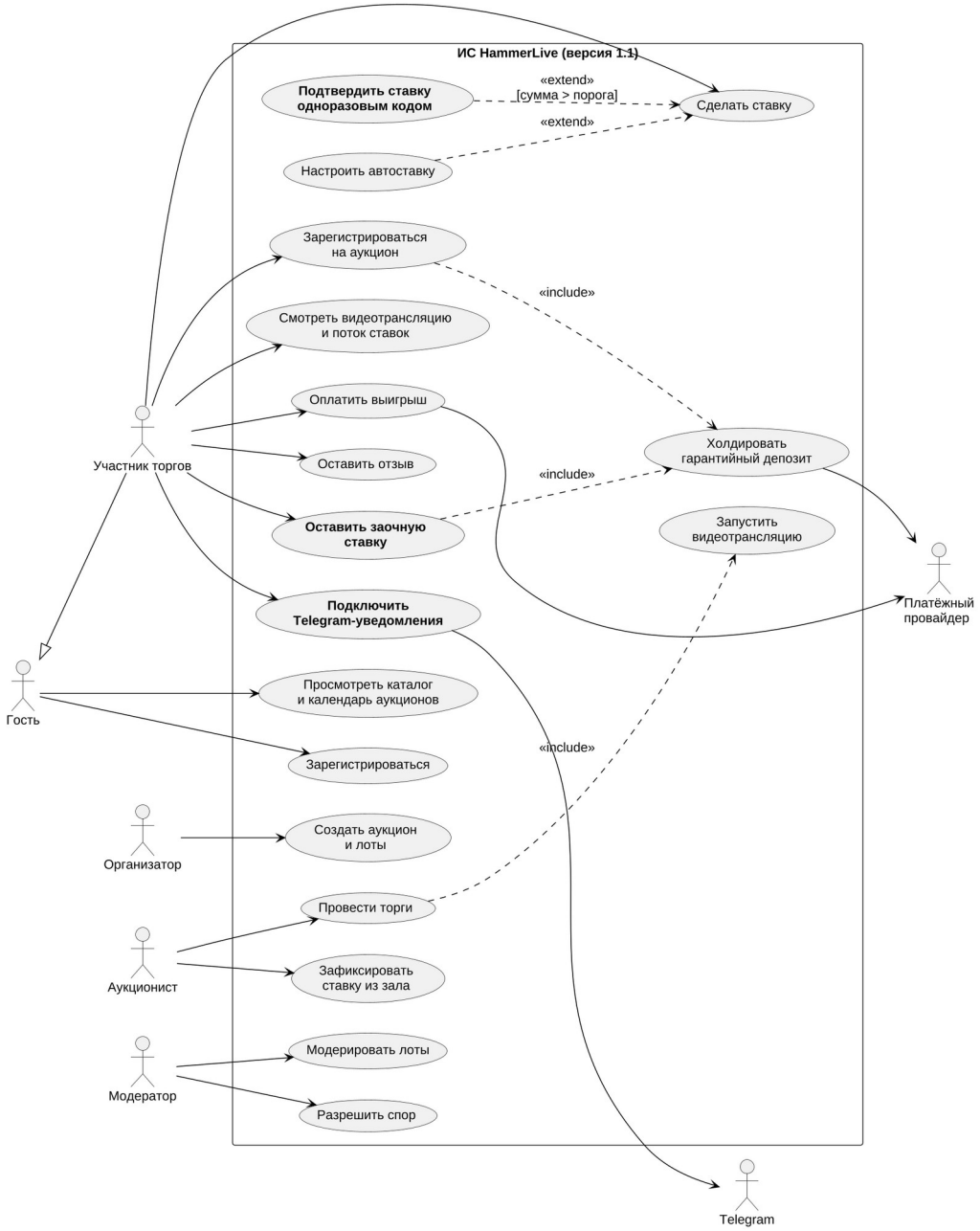


Рисунок 2 – Диаграмма вариантов использования (версия 1.1)

Из поведенческих диаграмм обновлена диаграмма последовательностей (рисунок 3): построен сценарий совершения крупной ставки с подтверждением. Фрагмент alt показывает отказ BidCapture с кодом STEP_UP_REQUIRED, отправку одноразового кода сервисом Identity и повтор команды с токеном подтверждения, который BidCapture проверяет в Identity. Только после этого ставка помещается в очередь, маршрутизируется по lotId в шард BidTracker, отвечающий за лот (FB-3), а результат приходит участнику событием bid.accepted от BidStreamer.

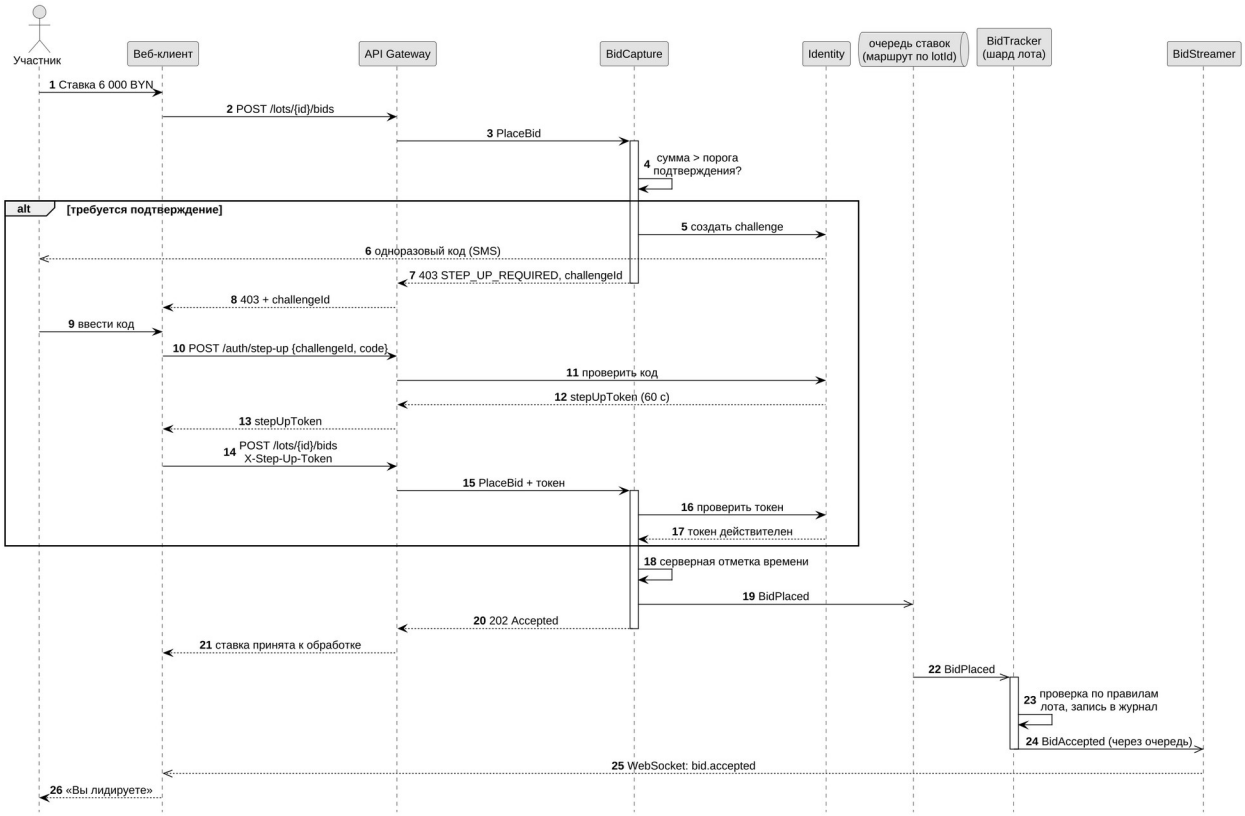


Рисунок 3 – Диаграмма последовательностей «Совершение крупной ставки с подтверждением»

2.3. Изменения архитектуры и API

Изменения архитектуры показаны на фрагменте диаграммы компонентов (рисунок 4). Заочные ставки (FB-1) принимает BidCapture через новый интерфейс IAbsenteeBids и передаёт в BidTracker через общую очередь ставок; BidTracker хранит их в таблице absentee_bids и при открытии лота превращает в автоставки, поэтому отдельный сервис не понадобился. Для FB-2 сервис Identity предоставляет интерфейсы IStepUp (выдача кода и токена клиенту) и IStepUpVerify (проверка токена сервисом BidCapture). Вспомогательный сервис Notification получает события лотов от BidTracker и доставляет уведомления через новый компонент TelegramAdapter (FB-4). Для выполнения FB-3 BidTracker разворачивается набором шардов, очередь ставок маршрутизирует сообщения по lotId, так что все ставки одного лота обрабатывает один шард, а BidCapture и BidStreamer масштабируются горизонтально.

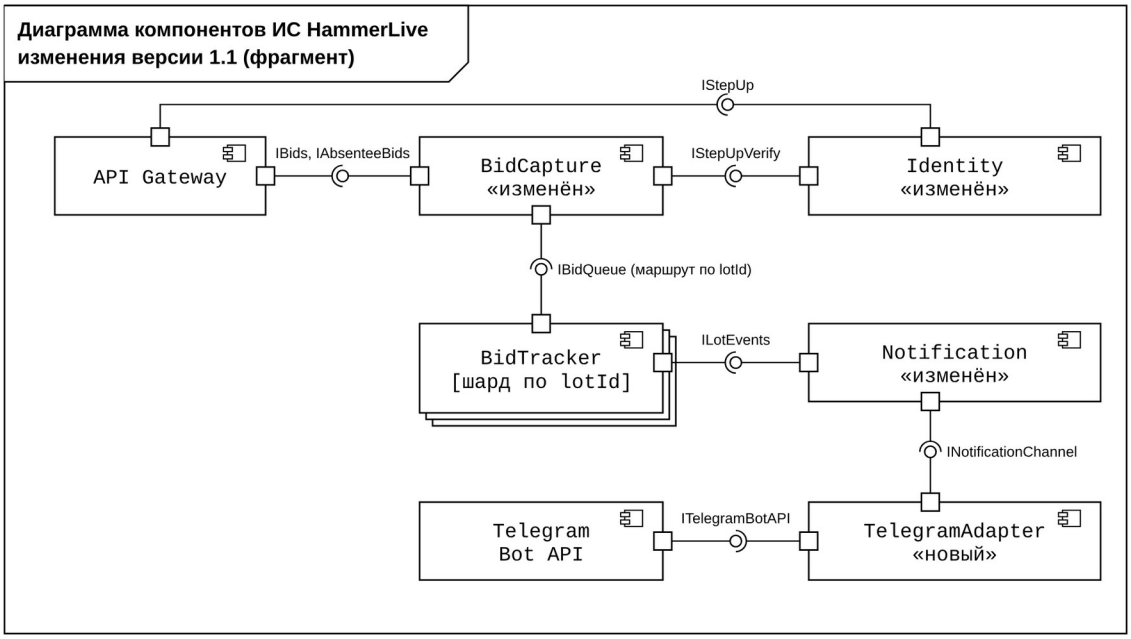


Рисунок 4 – Фрагмент диаграммы компонентов: изменения версии 1.1

Изменения спецификации API приведены в таблице 3; файл api/openapi.yaml обновлён до версии 1.1.0 и проверен валидатором OpenAPI.

Таблица 3 – Изменения API версии 1.1

Метод	Путь / элемент	Назначение	FB
POST	/lots/{lotId}/absentee-bids	Оставить заочную ставку	FB-1
DELETE	/lots/{lotId}/absentee-bids	Отменить заочную ставку	FB-1
POST	/auth/step-up	Проверить одноразовый код, выдать токен на 60 с	FB-2
–	Заголовок X-Step-Up-Token в POST /lots/{lotId}/bids	Подтверждение крупной ставки	FB-2
PUT, DELETE	/users/me/notification-channels/ telegram	Привязать или отключить Telegram	FB-4
–	STEP_UP_REQUIRED (HTTP 403), INVALID_OTP (HTTP 401), ABSENTEE_NOT_ALLOWED (событие bid.rejected)	Новые коды ошибок	FB-1, FB-2

В техническом задании изменены требования FR-USR-09 и FR-BID-10 (новые), FR-NTF-03 (приоритет «Ж» изменён на «О»), NFR-01 и NFR-02, а также критерии приёмки по производительности; названия сервисов в описании модулей ТЗ приведены в соответствие с диаграммой компонентов. В модель данных в базу BidTracker добавлена таблица absentee_bids, а для сервиса Notification выделена собственная база с таблицей notification_channels.

2.4. Версионирование и Pull Request

Все изменённые документы закоммичены в ветку feature/iteration-2-feedback тремя коммитами, в сообщении каждого указаны закрываемые замечания (таблица 1). Создан Pull Request №1 «Итерация 2: доработка документации по замечаниям FB-1 – FB-4 (v1.1)» в ветку main. В его описании для каждого замечания перечислены изменения в ТЗ, UML-диаграммах, API, модели данных и архитектуре, приведён список изменённых файлов и чек-

лист из шаблона .github/pull_request_template.md. Pull Request оставлен открытым для ревью преподавателем (заказчиком).

РАЗДЕЛ 3. КОМПЛЕКТ ПРОГРАММНОЙ И ЭКСПЛУАТАЦИОННОЙ ДОКУМЕНТАЦИИ

На основе документов ЛР №1–5 сформирован единый комплект документации версии 1.1. Он размещён в папке docs репозитория в формате PDF (исходник в формате Word – в docs/src). Состав комплекта приведён в таблице 4.

Таблица 4 – Состав комплекта документации

№	Документ	Источник	Стр.
–	Лист регистрации изменений	Замечания FB-1 – FB-4	3
1	Техническое задание (версия 1.1)	ЛР №2 + изменения итерации 2	4
2	Спецификация архитектуры и API (версия 1.1)	ЛР №3, №4 + изменения итерации 2	14
3	Набор UML-диаграмм (версия 1.1)	ЛР №3 + изменения итерации 2	21
4	Модель данных (ER-диаграммы)	ЛР №4 + изменения итерации 2	25
5	Руководство пользователя	Прототипы ЛР №3	28
6	Руководство системного администратора	ТЗ, архитектура ЛР №3, №4	31

Документы приведены к единому виду: общий титульный лист и оглавление, сквозная нумерация страниц, одинаковое оформление заголовков, текста и таблиц, нумерация рисунков и таблиц в пределах части (например, «Таблица 1.4», «Рисунок 2.1»). Изменённые по итогам итерации пункты помечены в тексте отметкой «изм. 1.1» со ссылкой на замечание. Руководство пользователя описывает три ключевых сценария – регистрацию на аукцион, участие в торгах и заочную ставку с оплатой выигрыша – и проиллюстрировано вайрфреймами из ЛР №3. Руководство системного администратора содержит требования к окружению, порядок развёртывания, переменные окружения, правила мониторинга и резервного копирования.

ЗАКЛЮЧЕНИЕ

Организация репозитория создаёт основу для успешной реализации HammerLive на следующих этапах. Разделение на папки docs, design, api, src и config позволяет каждому участнику команды быстро найти нужный материал, а хранение диаграмм и спецификации API в текстовом виде делает любые изменения проектных решений прозрачными и сравнимыми между версиями. README и шаблон Pull Request задают единые правила работы для новых разработчиков.

Работа через отдельные ветки и Pull Request снижает риски проекта: изменения не попадают в основную ветку без проверки, каждое из них связано с конкретным замечанием (FB-1 – FB-4) или требованием ТЗ, а история коммитов показывает, когда и почему изменилось решение. Это защищает от рассогласования документов, когда ТЗ, диаграммы и API описывают разные

версии системы, – в итерации 2 одно замечание затрагивало сразу несколько документов, и все они были обновлены в одном Pull Request.

Контроль качества на последующих этапах обеспечивают единый комплект документации с листом регистрации изменений, спецификация OpenAPI, проверяемая валидатором, заготовки тестов для каждого сервиса и описанное в docker-compose воспроизводимое окружение. Эти элементы позволят подключить автоматическую проверку в CI при каждом Pull Request и вести разработку итеративно, не теряя связи между требованиями, архитектурой и кодом.